

CAHIER DES CHARGES

Outil de Suivi des Mises en Production

Projet	Outil de Suivi MEP
Date	29/12/2024 (Mis à jour le 02/02/2026)
Type	Projet alternance

1. OBJECTIF DU PROJET

Mettre en place un outil de suivi des mises en production permettant de :

- Suivre les ordres de transport (OT) et leurs modifications
- Planifier et suivre les demandes de mise en production
- Rechercher et consulter toutes les demandes

Contexte actuel :

Actuellement, le suivi se fait via un fichier Excel.

Fonctionnalités principales :

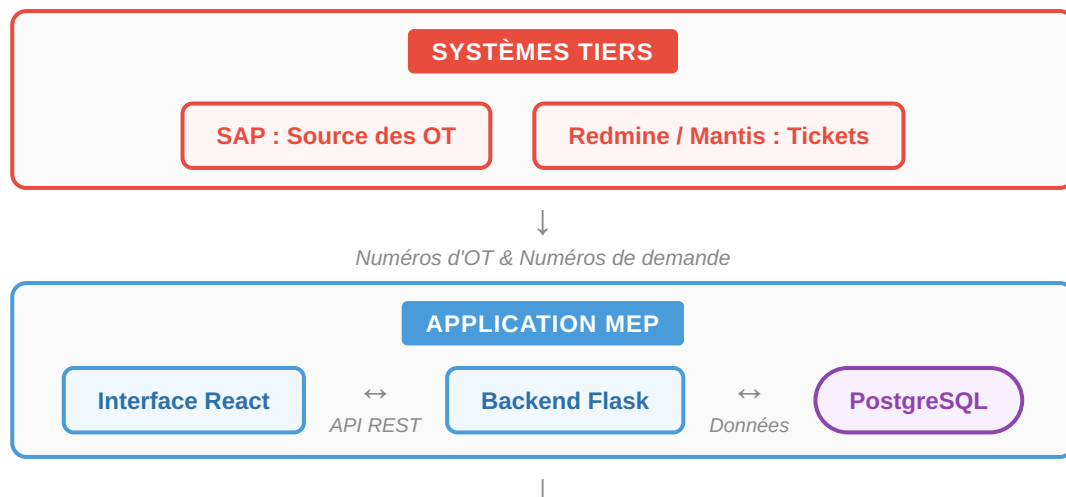
- Saisie d'un OT
- Indiquer que la mise en production (MEP) a eu lieu, avec qui et quand

2. CHAMPS DE DONNÉES

Champ	Description	Notes
Numéro de l'OT	Identifiant de l'ordre de transport	Obligatoire
Intitulé de l'OT (désignation)	Description de l'OT	Obligatoire
Titulaire de l'OT	Personne responsable	Obligatoire
Numéro de la demande (Mantis/Redmine)	Référence de la demande	Obligatoire
Lot de transport	Séquence à calculer pour chaque demande	Obligatoire
Ordre de passage	Ordre de traitement	Obligatoire
Date et heure souhaitées	Date/heure de mise en production	Obligatoire
Demandeur	Personne ayant créé la demande	Obligatoire
Remarque	Notes supplémentaires	Optionnel
MEP effectuée	Indiquer si la mise en production est faite, qui et quand	Obligatoire

3. DIAGRAMME DE FLUX DES ÉCHANGES

Schéma des échanges entre les systèmes tiers, l'application et l'infrastructure :





4. FRONTEND

4.1 Technologies

Technologie	Utilité
React	Framework JavaScript
CSS/Tailwind	Styles

4.2 Pages à créer

- Page de connexion
- Liste des OT avec recherche
- Formulaire de création/modification d'OT
- Détail d'un OT

5. BACKEND

5.1 Technologies

Technologie	Utilité
Python 3.x	Langage serveur
Flask	Framework web
PostgreSQL	Base de données

5.2 Gestionnaire de source

Utilisation impérative d'un gestionnaire de source basé sur **Git**, en l'occurrence **GitLab**, pour le versioning du code source (frontend et backend).

5.3 API REST

Routes à créer :

Méthode	Route	Action
POST	/api/auth/login	Connexion utilisateur
GET	/api/ots	Liste tous les OT
GET	/api/ots/<id>	Détails d'un OT
POST	/api/ots	Créer un OT
PUT	/api/ots/<id>	Modifier un OT
DELETE	/api/ots/<id>	Supprimer un OT

6. BASE DE DONNÉES

6.1 Système

PostgreSQL (développement et production)

6.2 Tables

Table : utilisateurs

Colonne	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
email	VARCHAR(100)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
nom	VARCHAR(100)	NOT NULL
prenom	VARCHAR(100)	NOT NULL
date_creation	TIMESTAMP	DEFAULT NOW()

Table : ordres_transport

Colonne	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
numero_ot	VARCHAR(50)	UNIQUE, NOT NULL
intitule	TEXT	NOT NULL
titulaire	VARCHAR(100)	NOT NULL
numero_demande	VARCHAR(50)	NOT NULL
lot_transport	INTEGER	NOT NULL
ordre_passage	INTEGER	NOT NULL
date_souhaitee	TIMESTAMP	NOT NULL, DEFAULT 18:00
demandeur	VARCHAR(100)	NOT NULL
emplacement_id	INTEGER	FOREIGN KEY → emplacements(id)
statut_id	INTEGER	FOREIGN KEY → statuts(id)
remarque	TEXT	NULL
mep_effectuee	BOOLEAN	DEFAULT FALSE
mep_effectuee_par	VARCHAR(100)	NULL
mep_date	TIMESTAMP	NULL
user_id	INTEGER	FOREIGN KEY → utilisateurs(id)

Table : emplacements

Table de référence des serveurs/environnements de déploiement disponibles.

Colonne	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
nom	VARCHAR(100)	UNIQUE, NOT NULL
description	TEXT	NULL

Table : statuts

Table de référence des statuts du cycle de vie d'un OT.

Colonne	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
libelle	VARCHAR(50)	UNIQUE, NOT NULL
couleur	VARCHAR(20)	NOT NULL

Table : commentaires

Notes et échanges ajoutés sur un OT au fil du temps (distinct du champ "remarque" statique).

Colonne	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
ot_id	INTEGER	FOREIGN KEY → ordres_transport(id)
user_id	INTEGER	FOREIGN KEY → utilisateurs(id)
contenu	TEXT	NOT NULL
date_creation	TIMESTAMP	DEFAULT NOW()

Table : historique_modifications

Journal d'audit : trace toutes les modifications apportées aux OT (qui, quoi, quand).

Colonne	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
ot_id	INTEGER	FOREIGN KEY → ordres_transport(id)
user_id	INTEGER	FOREIGN KEY → utilisateurs(id)
champ_modifie	VARCHAR(100)	NOT NULL
ancienne_valeur	TEXT	NULL
nouvelle_valeur	TEXT	NULL
date_modification	TIMESTAMP	DEFAULT NOW()

7. HÉBERGEMENT

Serveur d'entreprise (**serveur santé&cie**)

8. BESOINS MATÉRIELS (ENVIRONNEMENT DE TEST)

Élément	Requis	Pourquoi
CPU	4 vCPU	Permet d'exécuter simultanément le frontend React, l'API Flask et PostgreSQL sans ralentissement majeur.
RAM	8 Go	Évite la saturation mémoire lors des tests (navigateur, backend, base de données, dépendances).
Stockage	50 Go SSD	Capacité suffisante pour le code, la base de test, les logs, les sauvegardes et les dépendances.
Système d'exploitation	Linux LTS	Compatibilité et stabilité pour Flask, PostgreSQL et l'outillage de déploiement.
Base de données	PostgreSQL dédié test	Isolation des données de test et réduction du risque d'impact sur la production.
Réseau	Connexion stable + accès aux outils tiers	Nécessaire pour valider les échanges avec GitLab, SAP et Redmine/Mantis selon les cas d'usage.

9. LIVRABLES

- Code source (frontend + backend) sur **GitLab**
- Application fonctionnelle
- Documentation d'installation et schémas de flux
- Script SQL de création de la base